CSA02 – ASSIGNMENT 1

Grama Jyothi Rural Electricity Co-operative

Name: R RAKSHITHA

Register Number: 192525454

Course: CSA0209

GitHub Repository Link: https://github.com/RAKSHITHA-5656/CP_CSA0209

TASK 1 – Understand the Problem and Identify the Data (4 Marks)

1.1 Problem Analysis

Grama Jyothi calculates electricity bills using previous and current meter readings. The program calculates consumption, applies the correct tariff slabs, adds the fixed charge, and produces an accurate bill. It must also handle a 16-bit meter register rolling over from 65535 to 0.

Input Data

Data item	Description and justification
Household number	Unique identifier used to connect readings and the bill to the correct household.
Previous reading	The earlier meter reading, needed to calculate units consumed.
Current reading	The latest reading, used with the previous reading to calculate consumption.
Category	Identifies the consumer or tariff category if categories have different rates.
Tariff rates and fixed charge	Required to calculate the energy charge and final bill.

Output Data

Data item	Description and justification
Units consumed	Electricity used during the billing period.
Energy charge	Charge calculated using the slab rates.
Fixed charge	Fixed amount added to the energy charge.
Total bill	Final payable amount: energy charge plus fixed charge.

Constants

- SLAB1_UNITS = 100: the first 100 units are free.
- SLAB2_UNITS = 200: marks the end of the second slab.
- RATE1_PAISE = 0, RATE2_PAISE = 225, RATE3_PAISE = 450.
- FIXED_CHARGE_PAISE = 5000, equal to Rs 50.00.

Values and Correctness Rules

- Valid consumption must not be negative. A negative result may indicate incorrect rollover handling.
- Money can contain fractional rupees, so store it as integer paise to avoid floating-point rounding.
- District totals can be large with 800,000 households; use a sufficiently wide integer type and check for overflow.
- The total bill must equal the energy charge plus the fixed charge.
- The correct tariff must be applied to each slab, and bills must be accurate to the paisa.
- Invalid readings and categories should be rejected instead of silently producing an incorrect bill.

Assumptions

- The meter is a 16-bit unsigned counter with readings from 0 to 65535.
- The meter may roll over from 65535 to 0; readings are assumed to span at most one rollover.
- The first 100 units are free, units 101–200 cost Rs 2.25 each, and units above 200 cost Rs 4.50 each.
- The fixed charge is Rs 50 per billing period.
- All monetary calculations use integer paise.
- The examples use one common tariff; if categories have different tariffs, rates must be selected by category.

Alternative Concept: Store Readings as Strings

A reading such as 01530 can be stored as the string "01530". The program converts each digit into an integer before arithmetic. This preserves leading zeros but adds conversion work.

```
char reading[] = "01530";
int value = 0;
for (int i = 0; reading[i] != '\0'; i++) {
    value = value * 10 + (reading[i] - '0');
}
printf("%d\n", value);  /* 1530 */
```

Feature	Integer representation	String representation
Conversion time	No conversion needed	O(d), where d is the number of digits
Comparison	O(1) for fixed-width integers	O(d) worst case
Space per reading	Fixed size; commonly 2 bytes for 16-bit reading	Approximately d + 1 bytes including terminator
Arithmetic	Direct arithmetic	Convert before arithmetic
Leading zeros	Not preserved	Preserved

Conclusion: Integer storage is preferable for billing calculations. Strings are useful when the original text format or leading zeros must be preserved.

TASK 2 – Choose the Data Types (4 Marks)

2.1 Data Type Selection

Data item	Suggested type	Reason
Household number	unsigned int	Non-negative identifier.
Previous/current reading	typedef unsigned short meter_t	Suitable for a 16-bit register, commonly 0–65535.
Units consumed	unsigned int	Consumption is non-negative.
Category	enum	Represents a fixed set of categories.
Tariff rate	long or unsigned int	Stores integer paise rates.
Fixed charge	long	Stores the fixed charge in paise.
Energy charge / total bill	long long	Provides additional range for monetary totals.
District-wide total	long long	Can accumulate large totals, subject to overflow checks.

2.2 Typical Sizes and Ranges

Exact sizes depend on the compiler and platform. Common sizes are: char 1 byte; short and unsigned short 2 bytes; int and unsigned int 4 bytes; long 4 bytes on Windows and commonly 8 bytes on 64-bit Linux; long long 8 bytes; float 4 bytes; double 8 bytes. float usually has 6–7 decimal digits of precision and double about 15–16. Check your own system with sizeof, limits.h, and float.h.

```
#include <stdio.h>
#include <limits.h>
#include <float.h>
int main(void) {
    printf("char: %zu byte\n", sizeof(char));
    printf("short: %zu bytes\n", sizeof(short));
    printf("unsigned short: %zu bytes\n", sizeof(unsigned short));
    printf("int: %zu bytes\n", sizeof(int));
    printf("unsigned int: %zu bytes\n", sizeof(unsigned int));
    printf("long: %zu bytes\n", sizeof(long));
    printf("long long: %zu bytes\n", sizeof(long long));
    printf("float: %zu bytes\n", sizeof(float));
    printf("double: %zu bytes\n", sizeof(double));
    printf("USHRT_MAX = %u\n", (unsigned)USHRT_MAX);
    printf("INT_MAX = %d\n", INT_MAX);
    printf("UINT_MAX = %u\n", UINT_MAX);
    printf("LLONG_MAX = %lld\n", LLONG_MAX);
    printf("FLT_DIG = %d, DBL_DIG = %d\n", FLT_DIG, DBL_DIG);
```

```
    return 0;
}
```

```c
#include <stdio.h>
#include <limits.h>
#include <float.h>
int main(void) {
    printf("char: %zu byte\n", sizeof(char));
    printf("short: %zu bytes\n", sizeof(short));
    printf("unsigned short: %zu bytes\n", sizeof(unsigned short));
    printf("int: %zu bytes\n", sizeof(int));
    printf("unsigned int: %zu bytes\n", sizeof(unsigned int));
    printf("long: %zu bytes\n", sizeof(long));
    printf("long long: %zu bytes\n", sizeof(long long));
    printf("float: %zu bytes\n", sizeof(float));
    printf("double: %zu bytes\n", sizeof(double));
    printf("USHRT_MAX = %u\n", (unsigned)USHRT_MAX);
    printf("INT_MAX = %d\n", INT_MAX);
    printf("UINT_MAX = %u\n", UINT_MAX);
    printf("LLONG_MAX = %lld\n", LLONG_MAX);
    printf("FLT_DIG = %d, DBL_DIG = %d\n", FLT_DIG, DBL_DIG);
    return 0;
}
```

```
input
USHRT_MAX = 65535
INT_MAX = 2147483647
UINT_MAX = 4294967295
LLONG_MAX = 9223372036854775807
FLT_DIG = 6, DBL_DIG = 15

...Program finished with exit code 0
Press ENTER to exit console.
```

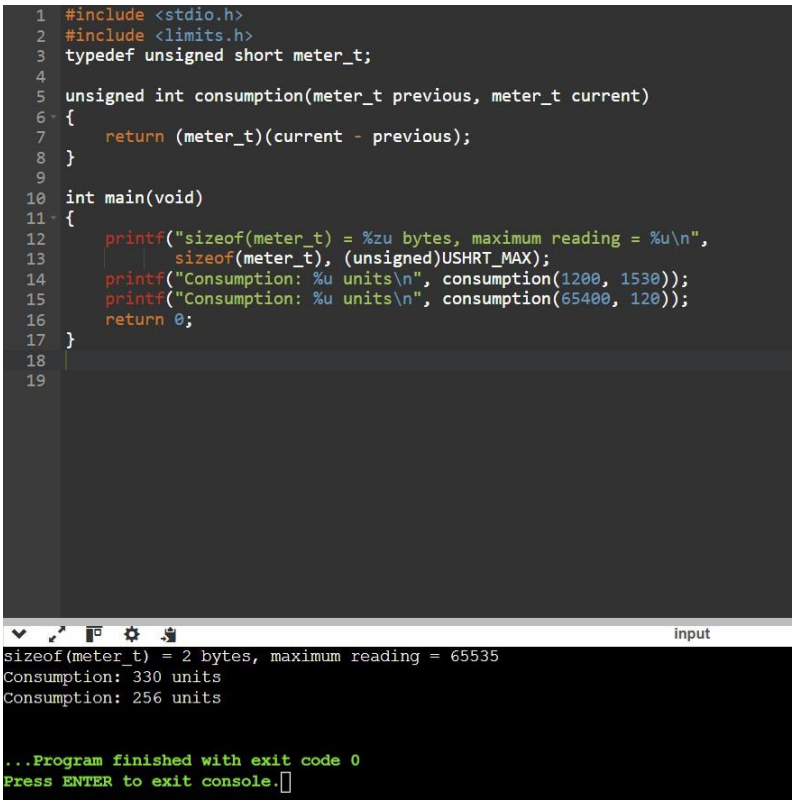
Compile: gcc -std=c11 -Wall -Wextra types.c -o types. Run with ./types (or types.exe on Windows). Record the actual output from your computer.

2.3 Complete the Code: Meter Rollover

```c
#include <stdio.h>
#include <limits.h>
typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
    return (meter_t)(current - previous);
}

int main(void)
{
    printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n",
           sizeof(meter_t), (unsigned)USHRT_MAX);
    printf("Consumption: %u units\n", consumption(1200, 1530));
    printf("Consumption: %u units\n", consumption(65400, 120));
    return 0;
}
```



Expected Output

```
sizeof(meter_t) = 2 bytes, maximum reading = 65535
Consumption: 330 units
Consumption: 256 units
```

For the first example, 1530 − 1200 = 330. For the rollover example, (65535 − 65400) + 1 + 120 = 256. Conversion to the unsigned 16-bit meter type applies arithmetic modulo 65536, so the result is not negative. This assumes no more than one rollover between readings.

2.4 Why Not Store Money as float?

```
#include <stdio.h>
int main(void)
{
    float total = 0.0f;
    long total_paise = 0;
    for (int i = 0; i < 4000; i++) {
        total += 0.1f;
        total_paise += 10;
    }
    printf("Float total: %.7f rupees\n", total);
    printf("Integer total: %ld paise\n", total_paise);
    printf("Integer total: Rs %ld.%02ld\n",
           total_paise / 100, total_paise % 100);
    return 0;
}
```

```c
#include <stdio.h>
int main(void)
{
    float total = 0.0f;
    long total_paise = 0;
    for (int i = 0; i < 4000; i++) {
        total += 0.1f;
        total_paise += 10;
    }
    printf("Float total: %.7f rupees\n", total);
    printf("Integer total: %ld paise\n", total_paise);
    printf("Integer total: Rs %ld.%02ld\n",
           total_paise / 100, total_paise % 100);
    return 0;
}
```

```
input
Float total: 400.0151978 rupees
Integer total: 40000 paise
Integer total: Rs 400.00


...Program finished with exit code 0
Press ENTER to exit console.
```

The integer calculation gives exactly 40000 paise, or Rs 400.00. A float may give a slightly different result because decimal fractions such as 0.1 cannot generally be represented exactly in binary floating-point.

Alternative Concept: double Rupees

Feature	Integer paise	double rupees
Addition time	O(1)	O(1)
Typical space	long long: 8 bytes	double: 8 bytes
Exact whole-paisa values	Yes, within integer range	Not guaranteed for every decimal fraction
Rounding	Usually not needed during calculation	May be needed when finalizing/displaying a bill
Accumulated error	No fractional rounding error if no overflow occurs	Small floating-point errors may accumulate

Adding n bills to a running total takes O(n) time and O(1) extra space. Storing all n bills requires O(n) space.

TASK 3 – Define the Constants (3 Marks)

3.1 Completed Billing Program

```c
#include <stdio.h>
#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000L
```

```
long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;
    if (units > SLAB2_UNITS) {
        paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE;
        units = SLAB2_UNITS;
    }
    if (units > SLAB1_UNITS) {
        paise += (long)(units - SLAB1_UNITS) * RATE2_PAISE;
    }
    /* The first 100 units are free. */
    return paise;
}

int main(void)
{
    int tests[] = {0, 100, 150, 200, 350};
    int n = sizeof tests / sizeof tests[0];
    for (int i = 0; i < n; i++) {
        long p = compute_bill(tests[i]);
        printf("%3d units -> Rs %ld.%02ld\n",
               tests[i], p / 100, p % 100);
    }
    return 0;
}
```

```
#include <stdio.h>
#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000L

long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;
    if (units > SLAB2_UNITS) {
        paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE;
        units = SLAB2_UNITS;
    }
    if (units > SLAB1_UNITS) {
        paise += (long)(units - SLAB1_UNITS) * RATE2_PAISE;
    }
    return paise;
}

int main(void)
{
    int tests[] = {0, 100, 150, 200, 350};
    int n = sizeof tests / sizeof tests[0];
    for (int i = 0; i < n; i++) {
        long p = compute_bill(tests[i]);
        printf("%3d units -> Rs %ld.%02ld\n",
               tests[i], p / 100, p % 100);
    }
```

input

```
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 162.50
200 units -> Rs 275.00
350 units -> Rs 950.00


...Program finished with exit code 0
Press ENTER to exit console.
```

Expected Output

```
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 162.50
```

```
200 units -> Rs 275.00
350 units -> Rs 950.00
```

3.2 Tariff Revision

Change #define RATE2_PAISE 225 to #define RATE2_PAISE 230, then recompile and run. Expected revised output:

```
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 165.00
200 units -> Rs 280.00
350 units -> Rs 955.00
```

3.3 #define, const, and enum

Feature	#define	const	enum
Purpose	Preprocessor replacement	Read-only object	Named integer constants
Type checking	No inherent type	Declared type	Integer constant
Scope	Preprocessor-controlled	C scope rules	C scope rules
Debugger visibility	Usually no separate object	May be visible as an object	Depends on compiler debug information
Storage	Macro itself uses no storage	Stored object may use memory	Enumerator values need no separate objects
Runtime overhead	None from replacement itself	Usually none for simple constant	None for enumerator itself

For tariff configuration, #define allows values to be revised in one place. const is useful for a typed read-only object. enum is suitable for named integer options such as consumer categories. Production code should validate input and check for overflow.

TASK 4 – Use Macros Safely (4 Marks)

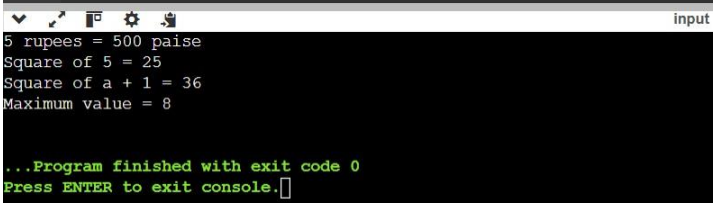
4.1 Program Using Macros

```
#include <stdio.h>
#define RUPEES_TO_PAISE(r) ((r) * 100)
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) (((a) > (b)) ? (a) : (b))

int main(void)
{
    int a = 5;
    int b = 8;
    printf("5 rupees = %d paise\n", RUPEES_TO_PAISE(5));
    printf("Square of 5 = %d\n", SQUARE(a));
    printf("Square of a + 1 = %d\n", SQUARE(a + 1));
    printf("Maximum value = %d\n", MAX(a, b));
    return 0;
}
```

```c
#include <stdio.h>
#define RUPEES_TO_PAISE(r) ((r) * 100)
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) (((a) > (b)) ? (a) : (b))

int main(void)
{
    int a = 5;
    int b = 8;
    printf("5 rupees = %d paise\n", RUPEES_TO_PAISE(5));
    printf("Square of 5 = %d\n", SQUARE(a));
    printf("Square of a + 1 = %d\n", SQUARE(a + 1));
    printf("Maximum value = %d\n", MAX(a, b));
    return 0;
}
```



Expected Output

```
5 rupees = 500 paise
Square of 5 = 25
Square of a + 1 = 36
Maximum value = 8
```

4.2 Macro Parentheses and Pitfalls

An unsafe definition, #define SQUARE(x) x * x, called as SQUARE(5 + 1), expands to 5 + 1 * 5 + 1 and evaluates to 11 because multiplication has higher precedence than addition. The safe definition #define SQUARE(x) ((x) * (x)) expands to ((5 + 1) * (5 + 1)) and returns 36.

MAX(a, b) can evaluate an argument more than once. For example, MAX(i++, j) expands with i++ in both the comparison and the selected expression. This can change i unexpectedly. Prefer a function when arguments may have side effects.

4.3 View Preprocessor Output

Run these commands in the folder containing macros.c:

```
gcc -E macros.c -o macros.i
cat macros.i       # Linux
 type macros.i     # Windows Command Prompt

# Or print preprocessed source directly:
gcc -E -P macros.c
```

The preprocessed output contains expressions similar to ((5) * 100), ((a + 1) * (a + 1)), and (((a) > (b)) ? (a) : (b)). Capture a screenshot of your own terminal output.

4.4 Alternative: static inline Functions

```
#include <stdio.h>
static inline int rupees_to_paise(int rupees) { return rupees * 100; }
static inline int square(int x) { return x * x; }
```

```
static inline int max_value(int a, int b) { return (a > b) ? a : b; }

int main(void)
{
    int a = 5, b = 8;
    printf("5 rupees = %d paise\n", rupees_to_paise(5));
    printf("Square of 5 = %d\n", square(a));
    printf("Square of a + 1 = %d\n", square(a + 1));
    printf("Maximum value = %d\n", max_value(a, b));
    return 0;
}
```

```
#include <stdio.h>
static inline int rupees_to_paise(int rupees) { return rupees * 100; }
static inline int square(int x) { return x * x; }
static inline int max_value(int a, int b) { return (a > b) ? a : b; }

int main(void)
{
    int a = 5, b = 8;
    printf("5 rupees = %d paise\n", rupees_to_paise(5));
    printf("Square of 5 = %d\n", square(a));
    printf("Square of a + 1 = %d\n", square(a + 1));
    printf("Maximum value = %d\n", max_value(a, b));
    return 0;
}
```

input

```
5 rupees = 500 paise
Square of 5 = 25
Square of a + 1 = 36
Maximum value = 8


...Program finished with exit code 0
Press ENTER to exit console.
```

Feature	Macro	static inline function
Time per simple calculation	Usually O(1)	Usually O(1)
Type checking	No inherent parameter type checking	Parameter types are checked
Argument evaluation	May evaluate arguments multiple times	Each argument expression is evaluated once per call
Code size	Expansion may duplicate code at each use	Compiler may reuse the body or inline it
Debugging	Can be harder	Usually easier

If a macro is used k times, expanded code can occupy O(k) total space for a fixed-size expression. A function may keep one copy of its body, although compiler optimization can change the final code size. static inline functions are generally safer for typed calculations and arguments with side effects.

GitHub Submission Checklist

- Create repository named CSA02-<RegisterNumber>-Assignment1.
- Create one folder for each task, such as Task01, Task02, Task03, and Task04.
- Include source code, input files if applicable, and actual output screenshots.
- Include README.md with name, register number, compiler/version, compile commands, run steps, and a short result note for each task.

- Make at least one commit for each task and push commits progressively.
- Share the repository with faculty as public or private with faculty added as a collaborator.
- Put the working GitHub link on the report cover page and in README.md.

Suggested README.md Template

```
# CSA02 – Assignment 1

## Student Details
- Name: [Your Name]
- Register Number: [Your Register Number]
- Course: CSA02
- Submission Date: 27-10-2026
- GitHub Repository: [Paste link]

## Compiler Details
- Compiler: GCC
- Version: [Output of gcc --version]
- Language: C

## Task 1 – Problem Analysis
Analysed inputs, outputs, constants, rules, assumptions, and string representation.

## Task 2 – Data Types
Tested type sizes, limits, integer paise, and meter rollover.
Compile: gcc -std=c11 -Wall -Wextra meter.c -o meter
Run: ./meter (Windows: meter.exe)

## Task 3 – Constants
Tested slab-based billing and tariff revision.
Compile: gcc -std=c11 -Wall -Wextra billing.c -o billing
Run: ./billing (Windows: billing.exe)

## Task 4 – Safe Macros
Tested macro expansion and compared macros with inline functions.
Compile: gcc -std=c11 -Wall -Wextra macros.c -o macros
Run: ./macros (Windows: macros.exe)
Preprocess: gcc -E -P macros.c

## Results
- Task 1: Analysed electricity billing data and rules.
- Task 2: Tested types and meter rollover.
- Task 3: Tested tariff slabs and tariff changes.
- Task 4: Tested safe macros and inline functions.
```

Final note: Replace all placeholders with your details. Run each program on your computer, verify the outputs, record your actual GCC version and sizeof results, and insert your own screenshots before submitting.